Puppet Expert Seminar

Day 1

Puppet Infrastructure Architecture, Environments, Code Deployment, and Operations

Day 1 — Agenda

Part 1 — Expert-Level Puppet Context

- Positioning Puppet in modern infrastructure automation
- Declarative configuration management principles
- Puppet compared with Ansible, Chef, SaltStack, Terraform, and Kubernetes-native approaches
- Typical enterprise Puppet architecture patterns

Part 2 — Deep Dive into Puppet Architecture

- Core components revisited at expert depth
- Agent-server communication lifecycle
- Catalog compilation process in detail
- Performance and scaling considerations

Part 3 — Designing a Scalable Puppet Infrastructure

- Single vs. multi-server designs
- High availability patterns
- Load balancing and certificate authority separation
- PuppetDB architecture and performance

Part 4 — Environments and Environment Strategy

- Environment isolation and lifecycle
- Feature branches, staging, per-team environments
- Common environment problems

Part 5 — Puppet Code Deployment with r10k

- Control repository concept
- Puppetfile and module management
- r10k configuration and deployment
- Integrating r10k into CI/CD pipelines

Part 6 — Monitoring and Operational Observability

- Monitoring Puppet agents, server, and PuppetDB
- Prometheus, Grafana, log aggregation
- Alerting strategies and reporting dashboards

Part 7 — Troubleshooting Puppet Infrastructure

- Certificate, environment, and catalog compilation problems
- Debugging tools and commands

Part 1 — Expert-Level Puppet Context

Where Puppet Fits Today

Puppet is one of several infrastructure automation tools. Understanding when to use it — and when not to — is a mark of expertise.

Tool	Paradigm	Primary strength
Puppet	Declarative DSL, agent-based	Long-term desired state enforcement, large Linux estates
Ansible	Imperative YAML, agentless SSH	Rapid ad-hoc automation, Windows, simple playbooks
Chef	Declarative/imperative, agent-based	Ruby-native teams, complex cookbook ecosystems

Tool	Paradigm	Primary strength
SaltStack	Declarative/imperative, agent or agentless	High-performance event-driven automation
Terraform	Declarative HCL	Cloud resource provisioning, immutable infra
Kubernetes operators	Declarative controllers	Container workload lifecycle management

> Puppet excels at **continuous enforcement** of OS-level configuration across hundreds or thousands of long-lived Linux servers.

Declarative Configuration Management — Expert View

What "desired state" really means:

- You describe the state the node *should* be in
- Puppet determines *what needs to change* on each run
- The agent applies only the delta — not a full rebuild
- Every run is a **convergence** toward the declared state

Idempotency at expert level:

- A resource applied twice must produce the same result as applied once
- Violating idempotency in custom types/providers is the #1 cause of production incidents
- `exec` resources are the most common idempotency trap — always use `unless`, `onlyif`, or `creates`

Drift detection:

- Puppet detects when the real world diverges from the catalog
- Every agent run reports `changed`, `unchanged`, `failed`, `skipped` for each resource
- Persistent drift → investigate root cause, not just re-run Puppet

Common Mistakes in Growing Puppet Installations

Mistake	Consequence	Better approach
No control repository	Code lives only on server; no history	Git from day one
One giant `site.pp`	Unmanageable node classification	Role/profile pattern
Data hardcoded in manifests	Cannot vary per environment	Move everything to Hiera
No module versioning	Forge update breaks production	Pin versions in Puppetfile
Single Puppet Server, no HA	One outage blocks all deployments	Plan for resilience early

Idempotency by Resource Type

Understanding *how* each core resource type achieves idempotency helps you debug unexpected behaviour:

Resource type	How idempotency is checked	Common pitfall
`file`	Compare MD5 checksum of content + metadata	Symlink vs. file ownership surprises
`package`	Query package manager (rpm/dpkg) for installed version	`ensure => latest` causes uncontrolled upgrades
`service`	Check `systemctl is-active` / `status` exit code	Service up but unhealthy — Puppet says "running"

Resource type	How idempotency is checked	Common pitfall
`user`	Inspect `/etc/passwd` and shadow	Shell or home dir change triggers a diff every run
`exec`	NOT idempotent by default — requires `unless` / `onlyif` / `creates`	Missing guard → command runs on every Puppet run
Custom type	Depends on `exists?` provider method returning accurate state	Incorrect `exists?` → creates resource that already exists

> **Rule:** If you cannot describe a clear `exists?` check for a resource, reconsider building a custom type.

Drift Detection — Report Status Categories

Every agent run produces a report. Each resource falls into one of four categories:

Status	Meaning	Action
changed	Resource was out of sync; Puppet corrected it	Investigate why drift occurred
unchanged	Resource was already in desired state	Normal — no action needed
failed	Puppet attempted to apply but encountered an error	Fix the error; check logs
skipped	Resource was not evaluated (dependency failed, or `noop`)	Check ordering and dependencies

```
# Query PuppetDB for all nodes with failed resources in last run
puppet query 'nodes[certname] { latest_report_status = "failed" }'

# Which specific resources failed on a node?
puppet query 'resources[certname, type, title] {
  certname = "web01.example.com" and
  latest_report_status = "failed"
}'
```

Part 2 — Deep Dive into Puppet Architecture

Core Components — Expert Map

Component	Binary / Port	Purpose
Puppet Server	`puppetserver` / `:8140`	JVM-based; compiles catalogs, serves files, hosts CA
JRuby pool	inside Puppet Server JVM	Runs Puppet Ruby code; pool size drives throughput
Puppet Agent	`puppet` / no port	Runs on every node; applies catalogs
PuppetDB	`puppetdb` / `:8081`	PostgreSQL-backed store for facts, catalogs, reports
Facter	`facter`	Collects system facts; ships with `puppet-agent` AIO

Component	Binary / Port	Purpose
Hiera	library	Hierarchical configuration data; part of Puppet
CA service	inside Puppet Server or separate	Signs/revokes certificates; manages CRL
File server	inside Puppet Server	Serves `puppet:///modules/…` URIs over HTTPS
Report processor	inside Puppet Server	Receives agent reports; ships to PuppetDB / log
ENC	external script / Foreman	Provides class list and top-scope variables for a node

The Full Agent Run Lifecycle

Agent request
+ facts (JSON)

Agent Run — Step by Step

1. **Agent wakes up** (scheduled or `puppet agent --test`)
2. **Facter runs** — collects all facts for this node
3. **Agent connects** to Puppet Server over mTLS (port 8140)
4. **Certificate checked** — server verifies agent cert; agent verifies server cert
5. **Facts submitted** — full fact set sent as JSON in the request
6. **Node classification** — server calls ENC or evaluates `site.pp` to get class list
7. **Hiera lookups** — server reads data files for each parameter of each class
8. **Catalog compiled** — server resolves all classes, parameters, relationships into a DAG
9. **Catalog sent** — compiled catalog (JSON) returned to agent over HTTPS
10. **Agent applies catalog** — resources applied in dependency order
11. **Report submitted** — agent sends change/failure report back to server → PuppetDB

Catalog Compilation — Under the Hood

```
Compilation phases:
  1. Node scope:     top-scope variables from ENC / site.pp
  2. Class loading:  all included/required classes loaded from module path
  3. Parameter resolution:
       a. Explicit parameter in manifest?  → use it
       b. Hiera APL lookup (classname::param)?  → use it
       c. Class default value?  → use it
       d. Undef?  → use it or fail if required
  4. Resource evaluation:  all resource declarations executed
  5. Relationship graph:  before/require/notify/subscribe resolved
  6. Cycle detection:  Puppet fails fast on dependency cycles
  7. Catalog serialisation:  JSON catalog built and returned
```

Performance considerations:

- Large numbers of classes → more Hiera lookups → slower
- Exported resources → PuppetDB query per `<<| |>>` collection
- Complex ERB/EPP templates with heavy logic → CPU on server
- Deep module chains with many `require` / `include` calls → cascading compile time

JRuby Pool — The Performance Lever

Puppet Server runs Puppet Ruby code inside a **JRuby pool**:

```
puppetserver.conf:
  jruby-puppet {
    max-active-instances: 4          # default: num_cpus - 1
    max-requests-per-instance: 100000
    compile-mode: off                # 'jit' or 'force' for hot paths
  }
```

Sizing rules of thumb:

Agents	Puppet Servers	JRuby instances per server
< 100	1	2–4
100–500	1–2	4–8
500–2000	2–4	4–8 each
> 2000	4+ (load-balanced)	4–8 each

> Each JRuby instance uses ~512 MB RAM. Always size JVM heap = `max-active-instances × 512 MB × 1.5`.

Part 3 — Designing a Scalable Puppet Infrastructure

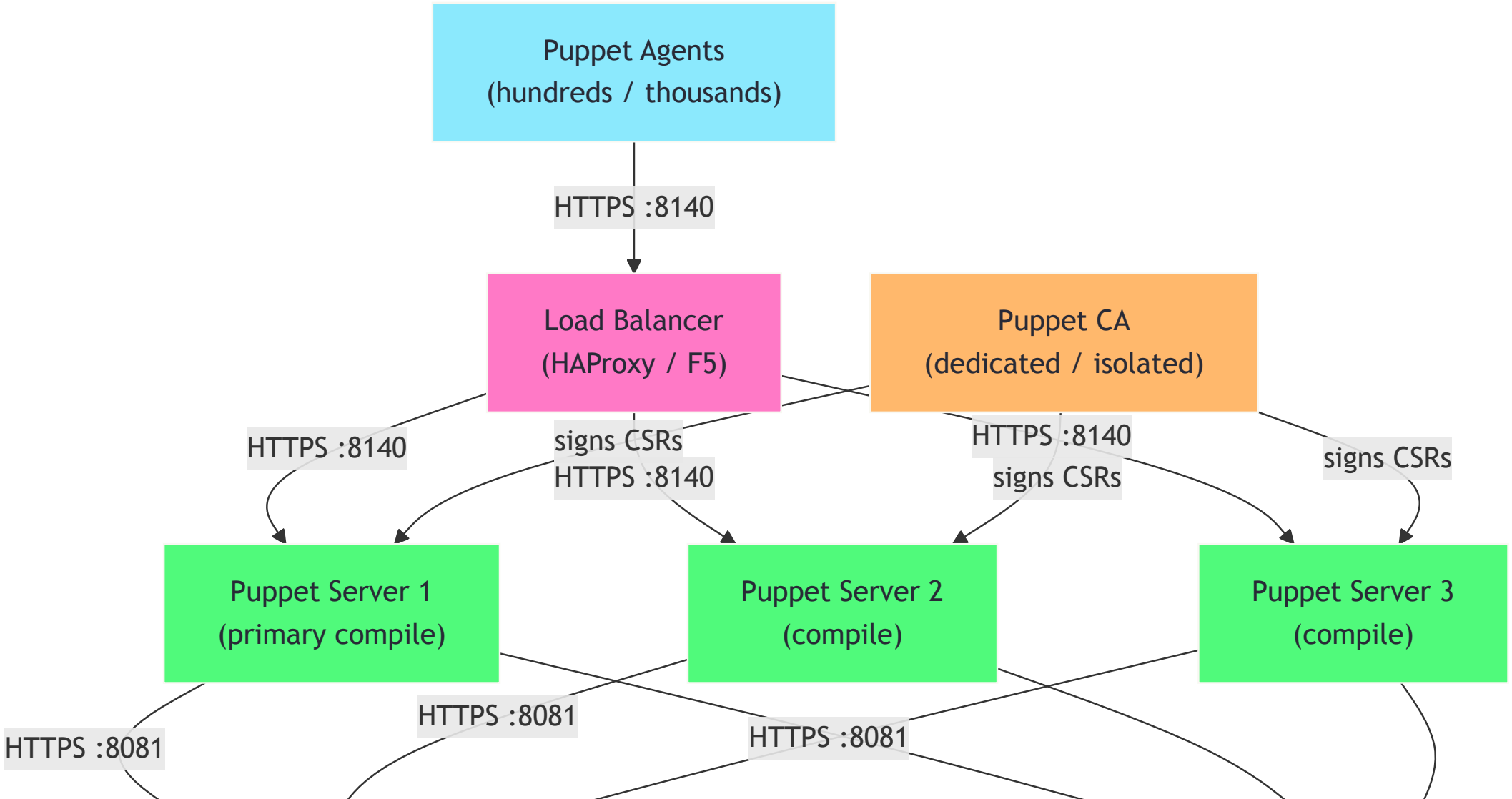
Infrastructure Sizing: A Decision Tree

```
How many agents?
  < 50        → Single Puppet Server (2 CPU, 4 GB) + PuppetDB (same host or separate)
  50–200      → Single Puppet Server (4 CPU, 8 GB) + PuppetDB (separate host)
  200–500     → Single Puppet Server (8 CPU, 16 GB) + PuppetDB (separate host, tuned PG)
  500–2000    → 2 compile servers + shared code volume + load balancer + dedicated CA
  > 2000      → 3–6 compile servers + load balancer + clustered PuppetDB + dedicated CA
```

What drives the decision:

- Catalog compile time (CPU-bound on JRuby)
- Number of concurrent agent connections
- PuppetDB write throughput (every agent run submits facts + report)
- Code deployment frequency

Scalable Puppet Infrastructure Architecture



Separating the Certificate Authority

In a scaled setup, the **CA should not be a compile server**:

```
Why separate the CA:
  - CA operations are low-frequency but security-critical
  - Load balancing compile servers behind a VIP breaks CA cert paths
    (agents see VIP IP, but cert CN is individual server hostname)
  - Dedicated CA can be locked down — no public exposure needed
  - CA restart does not affect compile capacity

How to set it up:
  1. Designate one host as CA server (ca.puppet.example.com)
  2. Compile servers: set certificate_authority_host = ca.puppet.example.com
  3. Load balancer: proxy :8140 to compile servers only
  4. Agents: server = puppet.example.com (LB VIP)
             ca_server = ca.puppet.example.com (direct)
```

Load Balancing Puppet Servers

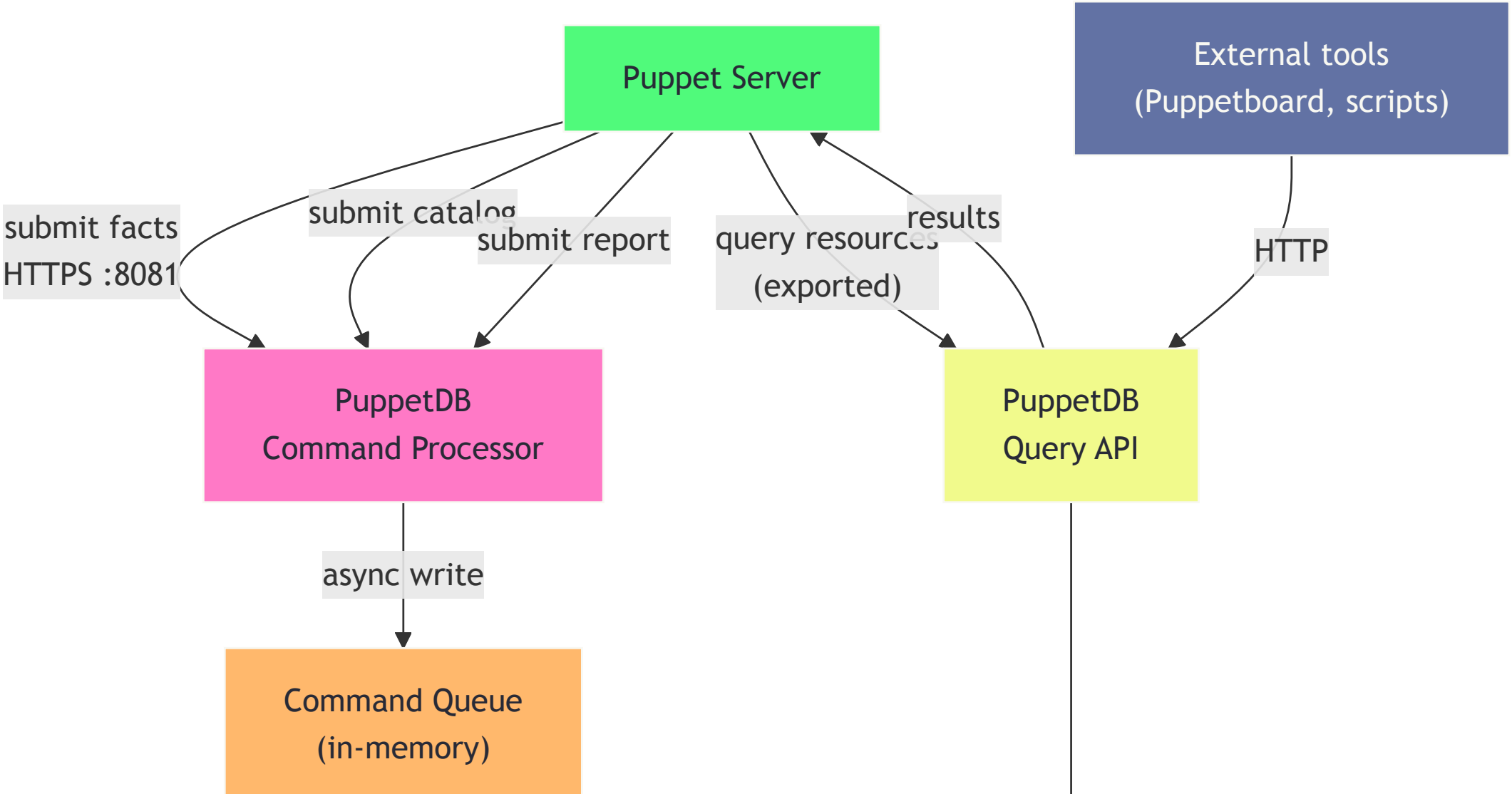
```
# Agent puppet.conf for load-balanced setup
[main]
server = puppet.example.com        # VIP / CNAME behind load balancer
ca_server = ca.puppet.example.com  # Direct CA — NOT behind LB

[agent]
runinterval = 1800
```

Load balancer considerations:

- Use **layer 4 (TCP) passthrough** — Puppet does its own mTLS
- Do NOT terminate SSL at the load balancer
- Use **least connections** balancing (catalogs vary in compile time)
- Health check: TCP connect to `:8140` or HTTP `GET /status/v1/simple`
- No sticky sessions needed — each catalog request is independent

PuppetDB Architecture Deep Dive



PuppetDB Performance Tuning

```
# /etc/puppetlabs/puppetdb/conf.d/database.ini
[database]
subname = //localhost:5432/puppetdb
username = puppetdb
password = <strong-password>
maximum-pool-size = 25        # match PostgreSQL max_connections budget
conn-max-age = 60
```

```
# /etc/default/puppetdb
JAVA_ARGS="-Xms1g -Xmx4g -XX:+UseG1GC"
```

```
-- PostgreSQL tuning (/etc/postgresql/*/main/postgresql.conf)
shared_buffers = 2GB            -- 25% of RAM
effective_cache_size = 6GB      -- 75% of RAM
work_mem = 64MB
maintenance_work_mem = 512MB
checkpoint_completion_target = 0.9
wal_buffers = 64MB
max_connections = 100
```

> PuppetDB is almost always the **bottleneck** in large environments. Size PostgreSQL first.

PuppetDB Data Retention

```
# /etc/puppetlabs/puppetdb/conf.d/database.ini
[database]
report-ttl = 14d          # delete reports older than 14 days
node-ttl = 7d             # mark nodes inactive after 7 days of silence
node-purge-ttl = 14d      # purge inactive nodes after 14 more days
gc-interval = 60          # run garbage collection every 60 minutes
```

Puppet Query Language (PQL) — operational queries:

```
# All nodes that failed in the last 24 hours
puppet query 'nodes[certname] { latest_report_status = "failed" }'

# Find all nodes that have a specific resource
puppet query 'resources[certname, title] { type = "Package" and title = "nginx" }'

# Facts for all nodes in a datacenter
puppet query 'facts[certname, value] { name = "datacenter" and value = "ams01" }'

# Nodes whose catalog changed in the last run
puppet query 'nodes[certname] { latest_report_noop = false and latest_report_hash != latest_report_hash }'
```

PQL — Operational Query Reference

PQL (Puppet Query Language) runs directly against PuppetDB. Essential queries for day-to-day operations:

```
# Count of nodes by Puppet agent version
puppet query 'nodes[count(), agent_version] { group by agent_version }'

# All packages managed by Puppet on a specific node
puppet query 'resources[title, parameters] {
  type = "Package" and certname = "db01.example.com"
}'

# Nodes that have NOT checked in for over 48 hours
puppet query 'nodes[certname, report_timestamp] {
  report_timestamp < "2026-04-30T00:00:00Z"
}'

# Find all nodes where a fact value is in a set
puppet query 'inventory[certname, facts.os.release.major] {
  facts.osfamily = "RedHat" and facts.os.release.major in ["7", "8"]
}'

# Catalog compile time (sorted, slowest first) — requires report data
puppet query 'reports[certname, metrics] {
  latest = true
  order by metrics.time.total desc
  limit 20
}'
```

Part 4 — Environments and Environment Strategy

What Environments Actually Are

A Puppet **environment** is a directory under `$environmentpath`:

```
/etc/puppetlabs/code/environments/
├── production/          ← default environment for all agents
│   ├── manifests/
│   │   └── site.pp
│   ├── modules/
│   ├── hiera.yaml
│   └── Puppetfile
├── staging/
│   └── ...
└── feature-add-ntp/
    └── ...
```

Agents choose their environment in `puppet.conf`:

```
[agent]
environment = staging
```

Or Puppet Server can override it via the ENC return value.

Environment Design Patterns

Pattern	When to use
production only	Tiny team, no testing capacity — acceptable for < 20 nodes
dev → staging → production	Standard pattern; staging mirrors production data/topology
feature branches	One environment per Git branch; test in isolation before merge
per-team environments	Large org: team-a-production, team-b-production; avoids contention
per-customer / per-tenant	MSP or SaaS: each customer is an isolated environment

Environment Caching — The Hidden Footgun

Puppet Server caches environment configuration. This is **not always obvious**:

```
# puppet.conf on Puppet Server
[server]
environment_timeout = 0          # Never cache (development only — kills performance)
environment_timeout = unlimited  # Cache forever (production — requires flush to update)
environment_timeout = 5m         # Cache for 5 minutes (balance for active development)
```

Flushing the environment cache:

```
# Flush all environments (safe — no agent disruption)
curl -sk -X DELETE \
  --cert /etc/puppetlabs/puppet/ssl/certs/puppet.example.com.pem \
  --key /etc/puppetlabs/puppet/ssl/private_keys/puppet.example.com.pem \
  https://localhost:8140/puppet-admin-api/v1/environment-cache

# Flush a specific environment
curl -sk -X DELETE \
  "https://localhost:8140/puppet-admin-api/v1/environment-cache?environment=staging" \
  --cert ... --key ...
```

> After `r10k deploy`, always flush the environment cache — or code changes will not be visible.

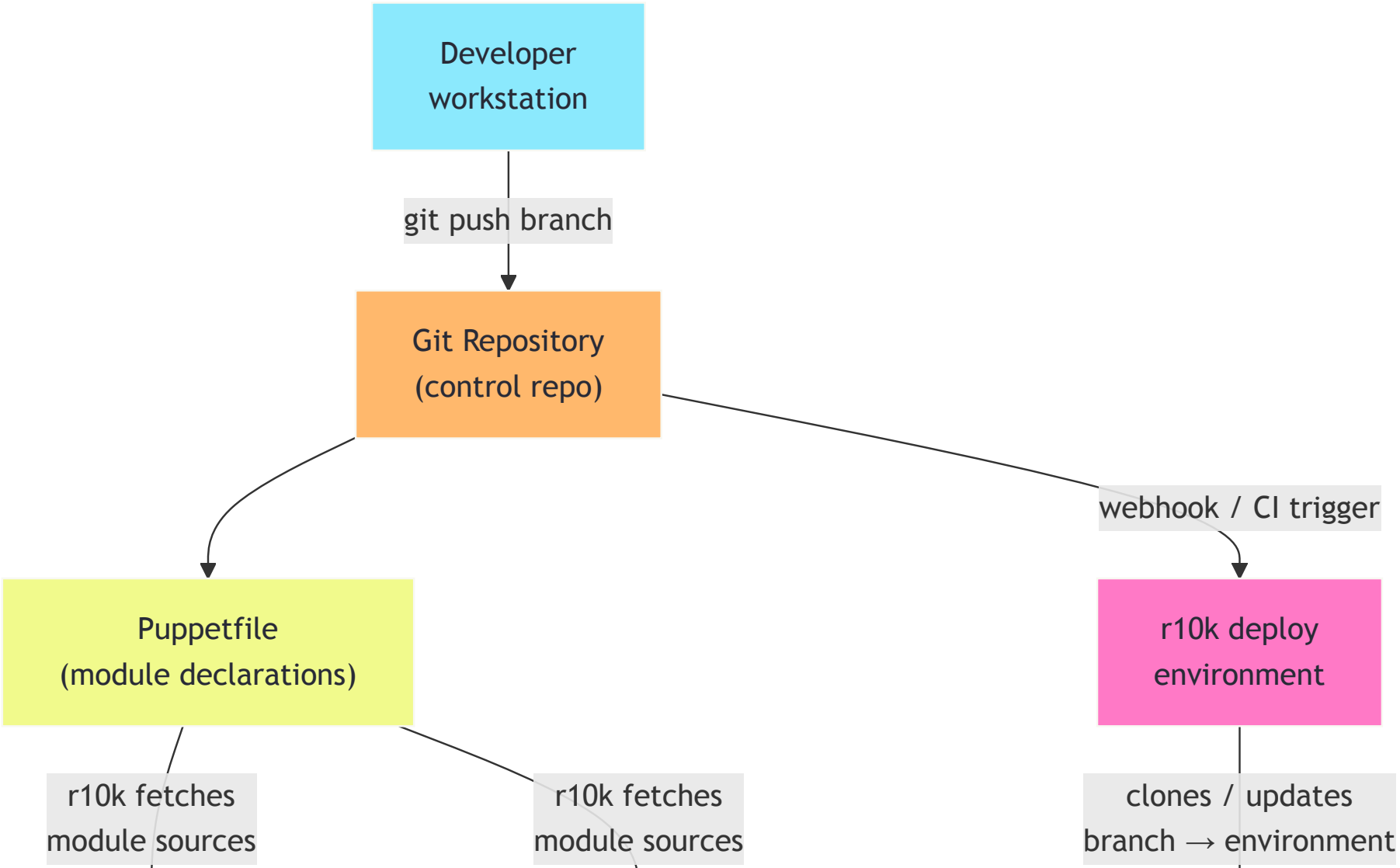
Part 5 — Puppet Code Deployment with r10k

The Control Repository — Your Infrastructure's Source of Truth

```
control-repo/
├── Puppetfile            ← declares ALL modules (Forge + internal)
├── hiera.yaml            ← environment-level Hiera configuration
├── manifests/
│   └── site.pp           ← node classification
├── data/                 ← environment-level Hiera data
│   ├── common.yaml
│   ├── nodes/
│   ├── roles/
│   └── os/
├── site-modules/         ← role/profile modules — live in this repo
│   ├── role/
│   └── profile/
└── .gitlab-ci.yml        ← CI/CD pipeline
```

> **One control repo per Puppet infrastructure.** Its branches map directly to Puppet environments.

Control Repository — Data Flow



Puppetfile Syntax

```
# Puppetfile

# Puppet Forge modules — pin exact versions in production
mod 'puppetlabs-stdlib',           '9.4.1'
mod 'puppetlabs-concat',           '9.0.2'
mod 'puppetlabs-apache',           '12.1.0'
mod 'puppet-nginx',                '5.0.0'
mod 'puppetlabs-postgresql',       '10.2.0'
mod 'puppet-chrony',               '1.2.0'

# Internal module from Git — pin to tag for stability
mod 'myorg-appserver',
  :git    => 'git@gitlab.example.com:puppet/puppet-appserver.git',
  :tag    => 'v2.3.1'

# Internal module from Git — develop branch (use in feature environments only)
mod 'myorg-monitoring',
  :git    => 'git@gitlab.example.com:puppet/puppet-monitoring.git',
  :branch => 'develop'

# Local module — not fetched by r10k, lives in site-modules/
# (Do not declare here — r10k will ignore the site-modules/ directory)
```

r10k Configuration

```yaml
# /etc/puppetlabs/r10k/r10k.yaml
---
# Location of control repo — r10k branches → environments
sources:
  main:
    remote: 'git@gitlab.example.com:puppet/control-repo.git'
    basedir: '/etc/puppetlabs/code/environments'

# Cache directory for Git repos (speeds up deployments)
cachedir: '/var/cache/r10k'

# Deploy settings
deploy:
  purge_levels:
    - deployment          # remove environments for deleted branches
    - puppetfile          # remove modules removed from Puppetfile
  purge_whitelist:
    - '.resource_types'   # keep pre-generated type data

# Logging
logging:
  level: warn
```

r10k Deployment Commands

```
# Deploy all environments (full sync)
r10k deploy environment -p

# Deploy a single environment (faster for iterative development)
r10k deploy environment staging -p

# Deploy with verbose output (shows what changed)
r10k deploy environment production -p -v

# Check what r10k would do without making changes (dry-run)
r10k deploy environment -p --dry-run

# Deploy modules only (Puppetfile changes, no manifests changed)
r10k deploy environment production -p --modules

# Show current deployment status
r10k deploy display
```

After r10k — flush the Puppet Server environment cache:

```
puppet-code deploy production --wait   # If using Puppet Code Manager
# Or manually:
curl -sk -X DELETE https://localhost:8140/puppet-admin-api/v1/environment-cache \
  --cert /etc/puppetlabs/puppet/ssl/certs/$(puppet config print certname).pem \
  --key /etc/puppetlabs/puppet/ssl/private_keys/$(puppet config print certname).pem
```

Git Branching Strategy for Puppet

```
main branch        → production environment
staging branch     → staging environment
develop branch     → development environment
feature/* branches → ephemeral test environments

Workflow:
  1. Create feature/add-ntp branch
  2. r10k auto-deploys feature/add-ntp environment
  3. Test: puppet agent --test --environment feature-add-ntp (on test node)
  4. Open Merge Request → automated CI (lint + unit tests)
  5. Review → merge to develop → merge to staging → merge to main
  6. r10k deploys main → production environment updated
  7. Flush environment cache
```

> **Never push untested code to `main`**. A broken catalog in production means all agents fail their next run.

CI/CD Pipeline for Puppet Code

```yaml
# .gitlab-ci.yml
stages:
  - validate
  - test
  - deploy-dev
  - deploy-staging
  - deploy-production

validate:
  stage: validate
  script:
    - pdk validate --parallel

unit-test:
  stage: test
  script:
    - pdk test unit --parallel

deploy-development:
  stage: deploy-dev
  script:
    - r10k deploy environment develop -p -v
    - puppet-code deploy develop --wait
  only: [develop]

deploy-staging:
  stage: deploy-staging
  script:
    - r10k deploy environment staging -p -v
    - puppet-code deploy staging --wait
  only: [staging]
  when: manual

deploy-production:
  stage: deploy-production
  script:
    - r10k deploy environment production -p -v
    - puppet-code deploy production --wait
  only: [main]
  when: manual
```

Part 6 — Monitoring and Operational Observability

What You Need to Monitor

```
Infrastructure health checklist:
  ✓ Puppet Server  — is the service up? compile performance? JRuby pool?
  ✓ PuppetDB       — is the service up? command queue? DB performance?
  ✓ PostgreSQL     — connections, replication lag, table bloat
  ✓ Agent runs     — last run time, run duration, changed resources, failures
  ✓ Certificates   — expiry dates, CSR queue
  ✓ Code sync      — r10k last run, environment freshness
  ✓ Report age     — any nodes that have not reported recently
```

Puppet Server Metrics — Prometheus Endpoint

Puppet Server exposes metrics via the `status` and `metrics` APIs:

```
# Service health check
curl -sk https://localhost:8140/status/v1/services \
  --cert /etc/puppetlabs/puppet/ssl/certs/$(hostname -f).pem \
  --key /etc/puppetlabs/puppet/ssl/private_keys/$(hostname -f).pem \
  | python3 -m json.tool

# JRuby pool metrics
curl -sk https://localhost:8140/status/v1/services/jruby-metrics \
  --cert ... --key ... | python3 -m json.tool

# Graphite / Prometheus metrics (requires puppetserver.conf config)
# Add to puppetserver.conf:
# metrics: {
#   server-id: puppet.example.com
#   registries: {
#     puppetserver: { reporters: { graphite: { enabled: true } } }
#   }
# }
```

Monitoring Agent Run Status

```
# PuppetDB: all nodes with their last run status
puppet query 'nodes[certname, report_timestamp, latest_report_status] {}'

# Nodes that have NOT reported in 3 hours (potential agent failure)
puppet query 'nodes[certname, report_timestamp] {
  report_timestamp < "2026-05-02T12:00:00Z"
}'

# All nodes that failed in last run
puppet query 'nodes[certname, latest_report_status] {
  latest_report_status = "failed"
}'

# Nodes with changed resources (non-noop run)
puppet query 'reports[certname, receive_time, noop_pending] {
  noop = false and corrective_change = true
  order by receive_time desc
  limit 20
}'
```

Monitoring with Prometheus and Grafana

Recommended Prometheus exporters for Puppet:

```yaml
# prometheus.yml scrape targets
scrape_configs:
  - job_name: puppet_server
    scheme: https
    tls_config:
      cert_file: /etc/prometheus/puppet-client.crt
      key_file: /etc/prometheus/puppet-client.key
      insecure_skip_verify: true  # use CA cert in production
    static_configs:
      - targets: ['puppet.example.com:8140']
    metrics_path: /metrics/v2/metrics

  - job_name: puppetdb
    static_configs:
      - targets: ['puppetdb.example.com:9090']   # puppetdb-exporter sidecar
```

Grafana dashboard panels to build:

- JRuby free/requested/max over time
- Catalog compile time percentiles (p50, p95, p99)
- Agent run duration histogram
- Failed vs. changed vs. unchanged resource counts per hour
- PuppetDB command queue depth
- PostgreSQL connection count and replication lag

Part 7 — Troubleshooting Puppet Infrastructure

Troubleshooting Decision Tree

```
Agent cannot connect?
  └─ Check: puppet config print server
  └─ Check: nc -zv puppet.example.com 8140
  └─ Check: SSL cert errors in agent log
  └─ Check: puppetserver service running

SSL/Certificate error?
  └─ Certificate mismatch? → puppet ssl clean + re-request
  └─ CA cert mismatch?     → re-download /etc/puppetlabs/puppet/ssl/certs/ca.pem
  └─ CRL outdated?         → puppet ssl download_cert --ca-only

Catalog compilation fails?
  └─ Run: puppet agent --test --debug 2>&1 | less
  └─ Check: puppet parser validate manifests/site.pp
  └─ Check: puppet lookup --explain <key> --node <certname> --environment <env>
  └─ Check: /var/log/puppetlabs/puppetserver/puppetserver.log

Environment not updating after r10k deploy?
  └─ Check: r10k deploy display
  └─ Flush environment cache (see earlier slide)
  └─ Check: environment_timeout in puppet.conf
```

Essential Debugging Commands

```
# Full debug agent run — most verbose output available
puppet agent --test --debug 2>&1 | tee /tmp/puppet-debug.log

# Show effective configuration (resolved from all puppet.conf layers)
puppet config print all

# Manually compile catalog for a node (run on Puppet Server)
puppet catalog compile agent01.example.com \
  --environment production \
  --render-as json | python3 -m json.tool

# Debug Hiera lookup for a specific key and node
puppet lookup profile::nginx::worker_processes \
  --node agent01.example.com \
  --environment production \
  --explain

# List all classes applied to a node (from PuppetDB)
puppet query 'resources[title] {
  certname = "agent01.example.com" and type = "Class"
}'

# Check certificate status
puppet ssl verify
puppetserver ca list --all

# View JRuby pool status live
watch -n5 "curl -sk https://localhost:8140/status/v1/services/jruby-metrics \
  --cert ... --key ... | python3 -c 'import sys,json; d=json.load(sys.stdin);
  print(d[\"status\"][\"jruby-pool\"][\"num-free-jrubies\"],
  \"/\", d[\"status\"][\"jruby-pool\"][\"max-active-instances\"])'"
```

Certificate Troubleshooting

```
# On the agent — clean and re-request certificate
puppet ssl clean
puppet agent --test    # submits new CSR

# On the Puppet CA server — view pending CSRs
puppetserver ca list

# Sign a specific agent certificate
puppetserver ca sign --certname agent01.example.com

# Sign all pending CSRs (use with caution in production)
puppetserver ca sign --all

# Revoke a compromised certificate
puppetserver ca revoke --certname compromised-node.example.com

# Verify a certificate
openssl verify -CAfile /etc/puppetlabs/puppet/ssl/certs/ca.pem \
  /etc/puppetlabs/puppet/ssl/certs/agent01.example.com.pem

# Check cert expiry date
openssl x509 -in /etc/puppetlabs/puppet/ssl/certs/agent01.example.com.pem \
  -noout -dates
```

Analyzing Puppet Server Logs

```
# Live log tail (most useful during troubleshooting)
journalctl -fu puppetserver

# Filter for errors and warnings only
journalctl -u puppetserver | grep -E 'ERROR|WARN' | tail -50

# Find slow catalog compilations (> 30 seconds)
grep "Compiled catalog" /var/log/puppetlabs/puppetserver/puppetserver.log \
  | awk '{print $NF, $0}' | sort -rn | head -20

# Find compilation errors
grep -A5 "Failed to compile catalog" \
  /var/log/puppetlabs/puppetserver/puppetserver.log | tail -100

# PuppetDB logs
journalctl -u puppetdb | grep -E 'ERROR|WARN' | tail -50

# PostgreSQL slow queries (requires pg_stat_statements)
psql -U puppetdb puppetdb -c "
  SELECT query, mean_exec_time, calls
  FROM pg_stat_statements
  ORDER BY mean_exec_time DESC
  LIMIT 10;"
```

Day 1 — Summary

Architecture mastery:

- You can design a scalable, HA Puppet infrastructure
- You understand catalog compilation deeply — not as a black box
- You can size JRuby pools and tune PuppetDB

Operations mastery:

- You can operate r10k and a control repository in production
- You can set up monitoring for all Puppet components
- You can systematically troubleshoot agent, server, and compilation problems

Tomorrow — Day 2:

Advanced Puppet language features, Hiera for enterprise data management, professional module design, testing, and code quality practices.

Day 1 — Practical Exercises

Exercise 1: Puppet HA Infrastructure Design and PuppetDB Analysis

Exercise 2: Build a Control Repository and Deploy with r10k